LFX Mentorship - Term 3 2025

KubeSlice: Implement Custom Topology Definition for a Slice

Candidate Information:

1. **Name** : Sai Sanjay Kottakota
2. **Github** : sanjay7178
3. **Email** : saisanjay7660@gmail.com
4. **Twitter** : sanjay7178
5. **Slack Username** :
6. **University Course**: Integrated Master's in Computer Science and Engineering (Current)
7. **University**: VIT University
8. **Time Zone**: IST (Indian Standard Time)

How did you find out about our mentorship program?

I found out about the LFX mentorship program through one of my friends. On exploring the site,I saw the program listed under open applications.

Why are you interested in this program?

After reviewing the KubeSlice custom topology proposal, I recognized the critical importance of this feature for the community. Its implementation would significantly enhance KubeSlice's network flexibility and efficiency by resolving challenges related to topology-aware network configuration, optimized connection matrix building, and VPN role management across distributed clusters. This project offers invaluable learning opportunities in Kubernetes networking internals, multi-cluster connectivity patterns, and resource optimization. I'm particularly intrigued by implementing connection matrix algorithms to transform topology configurations into efficient hub-spoke models (O(n) from $O(n^2)$). This experience has solidified my conviction in cloud-native architectures, demonstrating how effective topology directly influences operational efficiency and cost optimization. I'm excited to help users define custom connectivity patterns while maintaining backward compatibility.

What experience and knowledge/skills do you have that are applicable to the program ?

I have been active in Open Source contributions for more than a year now .I was selected for an Open Source fellowship, specifically Google Summer of Code 2025, with the OpenScience Labs organization.I gained experience working with Docker Swarm and Podman, and developed a Docker plugin called swarm-external-secrets using Golang. . My participation in Hacktoberfest 2024 was instrumental in fostering my initial engagement with the open-source community.
I developed Kubebrowse, an academic open-source project focused on Kubernetes. This platform provides secure browser isolation with an integrated safe file viewer and phishing mitigation. Through this project, I gained in-depth knowledge of Kubernetes and Golang, including Kubernetes CRDs Controllers Operators , RBAC policies, orchestration of headless services , Istio service mesh , Falco, OTEL tracing and log monitoring with Fluentd. I also delved into the guacd protocol and efficient handling of distributed websockets using Go routines.

I've also been a significant contributor to Kubeslice, and my contribution previously on Kubetail, a CNCF project, has deepened my understanding of Kubernetes.

My Involvement in Kubeslice Project:

- **Slack:** Actively suggested and implemented improvements (username: Sanjay).
- **Examples Repository:**
  - **Bookinfo with mTLS (PR #46):** Implemented Istio Bookinfo with mTLS across two worker clusters, created secure multi-cluster configuration, and verified deployment on GKE via KubeSlice.
  - **Contour Ingress Gateway (PR #45):** Added production-ready Contour/Envoy ingress for Bookinfo, configured inter-cluster communication via OpenVPN, and provided detailed documentation.
- **CLI Improvements:**

    - **Cluster Health Monitoring (PR #90):** Developed `show-health` command for KubeSlice cluster status, enhancing observability.
- **Infrastructure Improvements:**
    - **Multi-Architecture Support (PRs #24, #8):** Added multi-architecture Docker build support for `cmd-nsmgr` and `cmd-exclude-prefixes-k8s`, improving cross-platform compatibility and implementing automated workflows.
- **Kubeslice cli Installation Bug (Issue #97)**
    - Identified critical installation failure with kubeslice-cli install --profile=minimal-demo on Kind
    - Documented worker node availability issues in kubeslice-system namespace
    - Discovered compatibility problems with GKE Autopilot clusters that lack default worker nodes
    - Provided detailed reproduction steps and logs to facilitate resolution

Project Proposal

Title : Implement Custom Topology Definition for a Slice (Issue : #253)
Description :

🔖 Feature description
KubeSlice currently uses a full-mesh topology for the slice overlay network, where every cluster connects to every other cluster in the slice. This results in unnecessary tunnel creation and resource consumption.

🎤 Pitch
This project proposes a topology-aware design, allowing users to define custom connectivity matrices for slices. Users can specify partial meshes. Additionally, the project will support configuring each cluster's VPN deployment type (client/server) to accommodate network constraints such as firewalls or NAT.

✌️ Solution
Extension of the Slice CRD to support custom topology and VPN role definitions.
Logic to establish tunnels based only on the defined connectivity matrix.
Support for various deployment topologies (full-mesh, partial mesh, hub-spoke).
Sample configurations and documentation on how to define and validate topologies.

Feature Overview

I'm excited to propose my implementation approach for the Custom Topology Definition feature . This feature will transform KubeSlice's networking capabilities by moving beyond the current full-mesh topology to support flexible connection patterns like hub-spoke and partial mesh, significantly improving resource efficiency and network design flexibility.

Implementation Approach:

Core API Extension

I propose extending the SliceConfig CRD with a new `topologyConfig` field:

```
// SliceConfigSpec extension
type SliceConfigSpec struct {
    // Existing fields...

    // TopologyConfig defines custom connectivity topology for the slice
    TopologyConfig *TopologyConfig `json:"topologyConfig,omitempty"`
}

// New topology configuration types
type TopologyConfig struct {
    // TopologyType defines the connectivity pattern
    TopologyType TopologyType `json:"topologyType,omitempty"`
    // CustomConnections defines explicit connections
    CustomConnections []ClusterConnection
`json:"customConnections,omitempty"`
    // HubCluster defines the hub cluster name for hub-spoke topology
    HubCluster string `json:"hubCluster,omitempty"`
    // ClusterVPNRoles defines VPN roles for each cluster
    ClusterVPNRoles []ClusterVPNRole `json:"clusterVpnRoles,omitempty"`
}

// Topology type enum
type TopologyType string

const (
    FULL_MESH    TopologyType = "full-mesh"
    PARTIAL_MESH TopologyType = "partial-mesh"
    HUB_SPOKE    TopologyType = "hub-spoke"
    CUSTOM       TopologyType = "custom"
```

```
)
```

Architecture Visualization

Connection Matrix Algorithm

The core of my implementation is a flexible connection matrix builder that generates the appropriate connections based on topology type:

```
func buildConnectionMap(clusterNames []string, topologyConfig
*TopologyConfig) map[Connection]struct{} {
    // Default to full mesh if no topology config is provided
    if topologyConfig == nil {
        return buildFullMeshConnections(clusterNames)
    }

    switch topologyConfig.TopologyType {
    case FULL_MESH, "":
        return buildFullMeshConnections(clusterNames)
    case HUB_SPOKE:
        return buildHubSpokeConnections(clusterNames,
topologyConfig.HubCluster)
    case PARTIAL_MESH, CUSTOM:
        return buildCustomConnections(topologyConfig.CustomConnections)
    default:
        return buildFullMeshConnections(clusterNames)
    }
}

func buildHubSpokeConnections(clusterNames []string, hubCluster string)
map[Connection]struct{} {
    connectionMap := make(map[Connection]struct{})

    // Validate hub cluster exists
    hubExists := false
    for _, cluster := range clusterNames {
        if cluster == hubCluster {
            hubExists = true
            break
        }
    }

    // Connect hub to all spokes
```

```
    if hubExists {
        for _, cluster := range clusterNames {
            if cluster != hubCluster {
                connectionMap[Connection{
                    source: hubCluster,
                    destination: cluster,
                }] = struct{}{}
            }
        }
    }

    return connectionMap
}
```

My implementation ensures backward compatibility by defaulting to full-mesh behavior when no topology configuration is provided. This allows for gradual adoption without disrupting existing deployments.

Changes in SliceConfig on how users will use

```
apiVersion: controller.kubeslice.io/v1alpha1
kind: SliceConfig
metadata:
  name: hub-spoke-slice
  namespace: kubeslice-controller
spec:
  clusters: ["hub-cluster", "spoke-1", "spoke-2", "spoke-3"]
  topologyConfig:
    topologyType: hub-spoke
    hubCluster: hub-cluster
    clusterVpnRoles:
      - clusterName: hub-cluster
        role: server
      - clusterName: spoke-1
        role: client
```

12-Week Implementation Plan

Week 1-2 (September 8 - September 21, 2025)

- Set up development environment and create feature branch
- Design and implement CRD extensions (TopologyConfig, ClusterConnection, etc.)
- Create unit tests for API types and validation
- Submit initial PR for API extensions

Week 3-4 (September 22 - October 5, 2025)

- Implement core connection matrix algorithms for different topology types
- Develop the full mesh connection builder (backward compatibility)
- Implement hub-spoke topology algorithm
- Write unit tests for connection matrix builders

Week 5-6 (October 6 - October 19, 2025)

- Implement custom/partial mesh connection builders
- Integrate VPN role configuration system
- Modify gateway creation process to utilize topology configurations
- Add integration tests for gateway creation

Week 7-8 (October 20 - November 2, 2025)

- Implement validation webhook for topology configurations
- Add validation logic for different topology types
- Create test suite for validation logic
- Implement error handling and fallback mechanisms

Week 9-10 (November 3 - November 16, 2025)

- Conduct end-to-end testing of all topology types
- Develop performance tests and benchmarks
- Fix identified bugs and edge cases
- Implement backward compatibility tests

Week 11-12 (November 17 - November 30, 2025)

- Create comprehensive documentation
- Develop user guides with examples for each topology type
- Create architectural diagrams and usage patterns
- Final code cleanup and submission of PR

Closing

I'm excited about the opportunity to implement this feature that will significantly enhance KubeSlice's scalability and flexibility. My background in distributed systems and networking makes me well-suited to tackle this project.

Thank you for considering my proposal.